

Syllabus

Unit I

Introduction to **HTML**, history of HTML, objective, basic structure of HTML, header tags, body tags, paragraph tags.

- Tags for **FORM** creation: **TABLE**, **FORM**, **TEXTAREA**, **SELECT**, **IMG**, **IFRAME**, **FIELDSET**, **ANCHOR**.
- Lists in HTML, introduction to **DIV** tag, **NAVBAR** design.
- Introduction to **CSS**: types, selectors and responsiveness of a web page.
- Introduction to **Bootstrap** download / linking, using classes of Bootstrap, understanding the **Grid System**.
- Introduction to **WWW**: protocols and programs, applications and development tools, web browsers, **DNS**, web hosting providers.
- Setting up **Windows / Linux / Unix** web servers, web hosting in the cloud, types of web hosting.

Unit II

- Introduction to **JavaScript** functions and events, Document Object Model traversing using JavaScript.
- Output system: **alert**, throughput, input box, **console**. Variables and arrays in JavaScript.
- Date and string handling in JavaScript.
- Manipulating CSS through JavaScript: **form validation**(required, length, pattern).
- Advanced JavaScript: combining HTML, CSS and JavaScript events and buttons, controlling your browser.
- Introduction to **AJAX**: purpose, advantages and disadvantages, AJAX-based web applications and alternatives of AJAX.
- Introduction to **XML**: uses, key concepts, **DTD** and schemas, **XSL**, **XSLT** and XSL elements; transforming with XSLT. Introduction to **XHTML**.
- JSON**: introduction, keys and values, types of values, arrays, objects.

How to use this question bank

Each question is presented with a clear, exam-style answer followed by a code example wherever a tag, selector or script is involved. Diagrams are added where a visual makes the concept easier to remember. Read the answer once, then try to reproduce the code example without looking at the page.

Unit I - Questions and Answers

Q1. What is HTML? Explain its history and objectives.

Answer

HTML (Hyper Text Markup Language) is the standard markup language used to create the structure of web pages. It is **not a programming language** but a **markup language** that uses predefined tags to describe the meaning of every piece of content shown by the browser.

History (key milestones)

- **1989-1991** - Sir **Tim Berners-Lee** at CERN proposes HTML and the WWW.
- **1995** - HTML 2.0, the first formal specification by IETF.
- **1997** - HTML 3.2 by W3C (tables, applets).
- **1999** - HTML 4.01 (separation of content and style).
- **2000** - **XHTML 1.0** - HTML rewritten as well-formed XML.
- **2014** - **HTML5** by W3C / WHATWG: <video>, <audio>, <canvas>, semantic tags, form types.

Objectives

- Provide a **standard structure** for documents on the web.
- Allow content to be linked using **hyperlinks**
- Be **platform independent**- any browser on any OS can render the same HTML.
- Support **multimedia**- images, audio, video, forms and interactive elements.
- Be the **base layer** for CSS (presentation) and JavaScript (behaviour).

Q2. Explain the basic structure of an HTML document with a diagram.

Answer

Every HTML5 document begins with a **<!DOCTYPE html>** declaration that tells the browser to use the HTML5 standard. The whole page is wrapped inside an **<html>** element which contains exactly two children: **<head>** (metadata) and **<body>** (visible content).

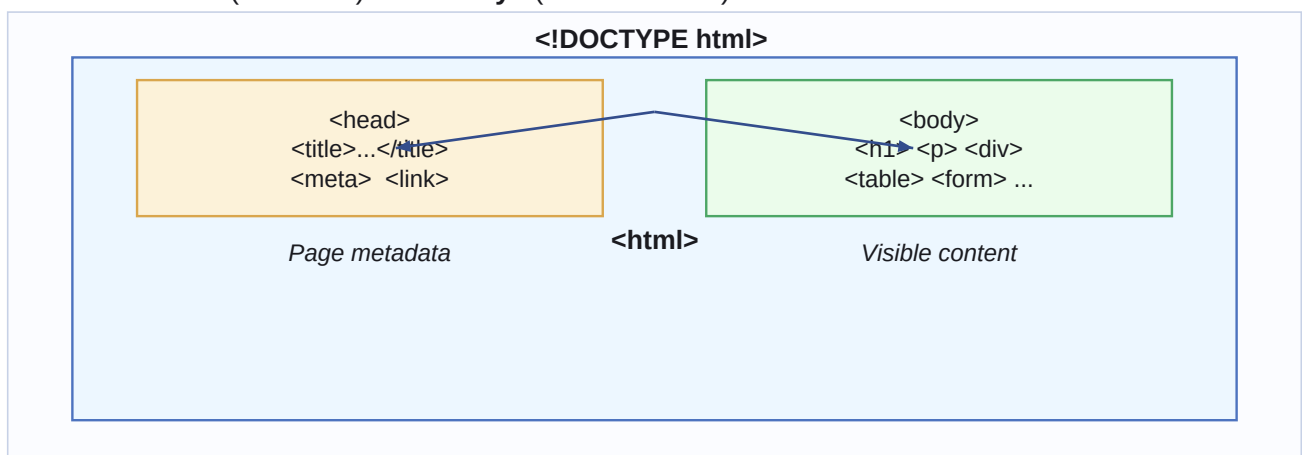


Figure 1.1 - Basic structure of an HTML5 document

Code example

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>My First Page</title>
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    <h1>Hello, World!</h1>
    <p>This is my first paragraph.</p>
  </body>
</html>
```

Q3. Differentiate between header tags, body tags and paragraph tags.

- **Header tags <h1> .. <h6>** define headings of decreasing importance. <h1> is the largest / most important; <h6> the smallest. They give the document its outline and help SEO.
- **Body tag <body>** is the container for everything visible on the page - text, images, links, forms, scripts, etc.
- **Paragraph tag <p>** marks a block of text as a single paragraph. The browser adds vertical space before and after a <p> by default.

```
<body>
  <h1>Web Technology</h1>      <!-- main heading -->
  <h2>Unit I</h2>              <!-- sub heading -->
  <p>HTML provides the structure of a web page.</p>
</body>
```

Q4. Explain the FORM, INPUT, TEXTAREA and SELECT tags with an example.

Answer

A **<form>** is a container that collects user input and sends it to a server. Its two key attributes are **action** (URL to send data to) and **method** (GET or POST). Inside a form we use **<input>** for single-line fields, **<textarea>** for multi-line text and **<select>** / **<option>** for drop-downs.

```
<form action="/register" method="post">
  <label>Name:</label>
  <input type="text" name="username" required>

  <label>Password:</label>
  <input type="password" name="pwd" minlength="6">

  <label>Course:</label>
  <select name="course">
    <option value="bca">BCA</option>
    <option value="mca">MCA</option>
  </select>

  <label>Address:</label>
  <textarea name="addr" rows="3" cols="30"></textarea>

  <input type="submit" value="Register">
</form>
```

Q5. Explain the TABLE tag with all its sub-tags. Draw the structure.

Answer

The **<table>** tag displays data in rows and columns. **<tr>** is a table row, **<th>** is a header cell (bold, centred by default) and **<td>** is a normal data cell. **<thead>**, **<tbody>** and **<tfoot>** group the head, body and footer of the table for clarity and CSS styling.

<table> --> <thead> + <tbody>

Roll	Name	Class	Marks
101	Aizaz	BCA-3	92
102	Ravi	BCA-3	88
103	Neha	BCA-3	95

<tr> contains <th> / <td>

Figure 1.2 - Logical structure of an HTML table

```
<table border="1" cellpadding="6">
  <thead>
    <tr><th>Roll</th><th>Name</th><th>Marks</th></tr>
  </thead>
  <tbody>
    <tr><td>101</td><td>Aizaz</td><td>92</td></tr>
    <tr><td>102</td><td>Ravi</td><td>88</td></tr>
  </tbody>
</table>
```

Q6. Explain IMG, IFRAME, FIELDSET and ANCHOR tags.

- **** embeds an image. The **alt** attribute provides text for screen readers and is shown if the image fails to load.
- **<iframe src="...">** - embeds another HTML page (e.g. a YouTube video, Google Map) inside the current page.
- **<fieldset>** with **<legend>** - groups related controls inside a form and draws a labelled border around them.
- **text** - the **anchor** tag creates a hyperlink. The **target="_blank"** attribute opens the link in a new tab.

```

<iframe src="https://example.com" width="400" height="250"></iframe>

<fieldset>
  <legend>Personal Details</legend>
  Name: <input type="text" name="n">
</fieldset>

<a href="https://www.google.com" target="_blank">Google</a>
```

Q7. Explain the different types of lists in HTML.

- **Ordered List ** - numbered list (1, 2, 3 ... or a, b, c via the type attribute).
- **Unordered List ** - bulleted list (disc, circle, square).
- **Description List <dl>** - pairs of <dt> (term) and <dd> (description), used for glossaries.

```
<ol>
  <li>HTML</li>
  <li>CSS</li>
  <li>JavaScript</li>
</ol>

<ul>
  <li>Mango</li>
  <li>Apple</li>
</ul>

<dl>
  <dt>HTML</dt>
  <dd>Hyper Text Markup Language - structure of a web page.</dd>
  <dt>CSS</dt>
  <dd>Cascading Style Sheets - presentation of a web page.</dd>
</dl>
```

Q8. What is the <div> tag and how is a NAVBAR designed using HTML and CSS?

Answer

<div> is a block-level **container** with no built-in meaning - it is used to group other elements so they can be styled or positioned together. A **navigation bar (navbar)** is a horizontal list of links placed at the top of the page; it is normally built with a <nav> wrapping a of <a> items, then styled with CSS Flexbox.

```
<nav class="navbar">
  <div class="brand">Tulas</div>
  <ul>
    <li><a href="#">Home</a></li>
    <li><a href="#">Courses</a></li>
    <li><a href="#">Contact</a></li>
  </ul>
</nav>

<style>
.navbar { display:flex; justify-content:space-between;
          background:#0d2a66; color:#fff; padding:12px 24px; }
.navbar ul { list-style:none; display:flex; gap:18px; }
.navbar a { color:#fff; text-decoration:none; }
.navbar a:hover { color:#ffb84d; }
</style>
```

Q9. What is CSS? Explain its types with the box model.

Answer

CSS (Cascading Style Sheets) describes how HTML elements should look on screen, paper or other media. It separates the **presentation** of a page from its **structure**, so the same HTML can be skinned in

many ways.

Three types of CSS

- **Inline CSS** - written inside a tag using the style attribute. Highest specificity but hard to maintain.
- **Internal / Embedded CSS** - placed inside a <style> block in the <head>. Affects only one page.
- **External CSS** - kept in a separate .css file linked with <link rel="stylesheet" href="style.css">. Best for large sites because one file styles many pages.

```
<!-- Inline -->
<p style="color:blue;">Hello</p>

<!-- Internal -->
<style> p { color: green; } </style>

<!-- External (in style.css) -->
p { color: red; font-size: 16px; }
```

CSS Box Model

Every element on a page is rendered as a rectangular **box** that has four layers, from inside to outside: **content**, **padding**, **border** and **margin**.

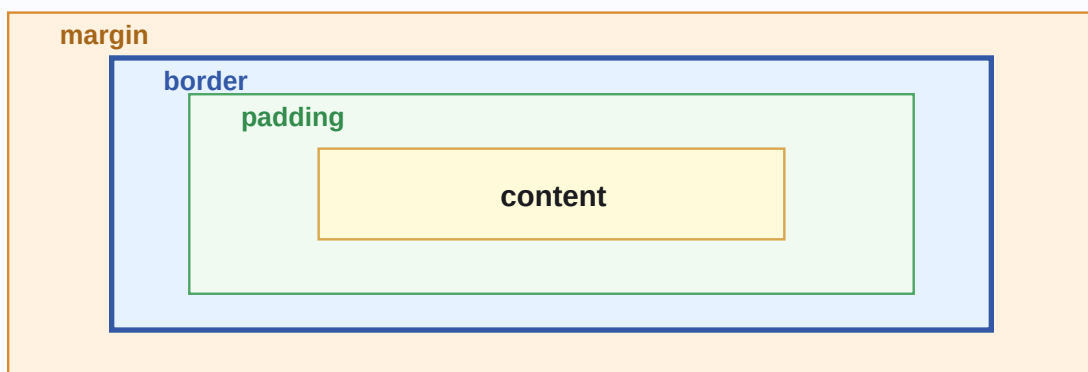


Figure 1.3 - The CSS Box Model

Q10. Explain CSS selectors with examples.

- **Universal selector *** - selects every element.
- **Type / element selector p** - every <p> tag.
- **Class selector .btn** - every element with class="btn".
- **ID selector #header** - the unique element with id="header".
- **Group selector h1, h2, p** - applies the same rule to many.
- **Descendant selector div p** - every <p> inside any <div>.
- **Pseudo-class a:hover** - state-based selector.
- **Attribute selector input[type="text"]** matches by attribute.

```
*          { box-sizing: border-box; }
body       { font-family: Arial, sans-serif; }
.btn       { background:#0d6efd; color:#fff; }
#main-title { text-align: center; }
a:hover    { text-decoration: underline; }
```

Q11. How do you make a web page responsive?

A **responsive** page automatically adapts its layout to the size of the device. Three pillars are used:

- **Viewport meta tag** - tells mobile browsers to use the device width: `<meta name="viewport" content="width=device-width, initial-scale=1">`.
- **Fluid units** - use percentages, em / rem, vw / vh and max-width instead of fixed pixels.
- **Media queries** - apply different CSS rules at different screen widths.

```
<meta name="viewport"
      content="width=device-width, initial-scale=1.0">

.container { width: 90%; max-width: 1100px; margin: auto; }

@media (max-width: 768px) {
  .container { width: 100%; }
  .menu      { display: none; }
}
```

Q12. What is Bootstrap? How is it linked to a project?

Bootstrap is the most popular open-source **front-end CSS framework** (originally developed at Twitter). It provides a ready-made library of CSS classes and JavaScript components (buttons, forms, modals, navbars, cards) that look the same across all modern browsers and adapt to every screen size.

Two ways to add Bootstrap

- **CDN (recommended for learning)** - one `<link>` for CSS and one `<script>` for JS bundle.
- **Local download** - download the compiled .css / .js files from getbootstrap.com and reference them with relative paths.

```
<!-- CDN method -->
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/
        dist/css/bootstrap.min.css" rel="stylesheet">

<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/
        dist/js/bootstrap.bundle.min.js"></script>
```

Q13. Explain the Bootstrap Grid System with a diagram.

Answer

Bootstrap divides every row into **12 invisible columns**. We build layouts by choosing how many columns each block should span on each device, using classes such as **col-sm-4**, **col-md-6**, **col-lg-3**. The numbers in one row should add up to 12.

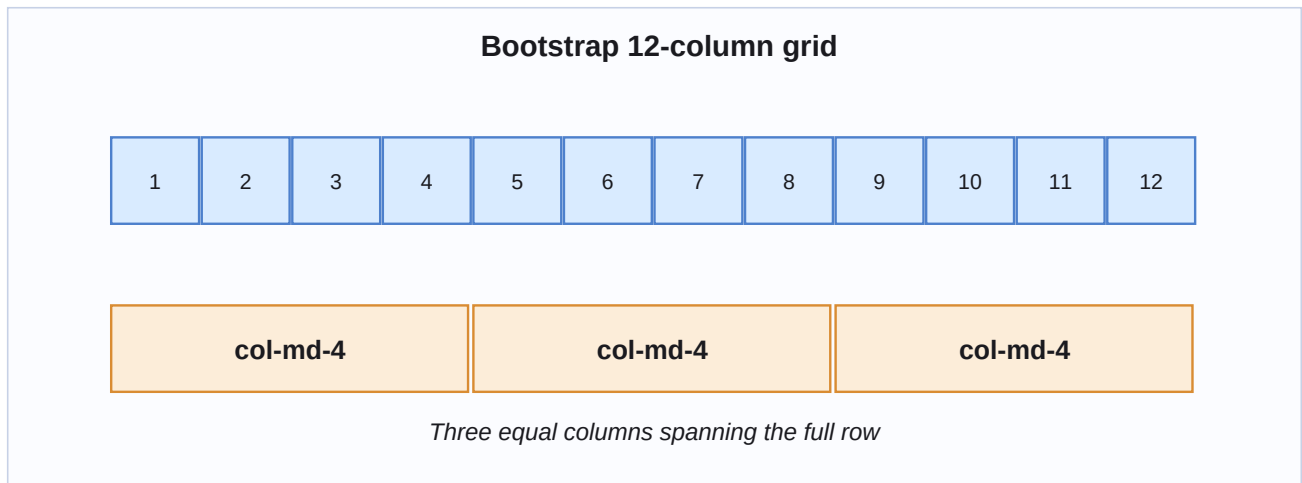


Figure 1.4 - Bootstrap 12-column grid (3 x col-md-4)

```
<div class="container">
  <div class="row">
    <div class="col-md-4">Column 1</div>
    <div class="col-md-4">Column 2</div>
    <div class="col-md-4">Column 3</div>
  </div>
</div>
```

Q14. What is WWW? Explain protocols, browsers, DNS and a web hosting provider

Answer

The **World Wide Web (WWW)** is a global system of inter-linked documents and other resources, identified by URLs and delivered over the Internet using HTTP / HTTPS.

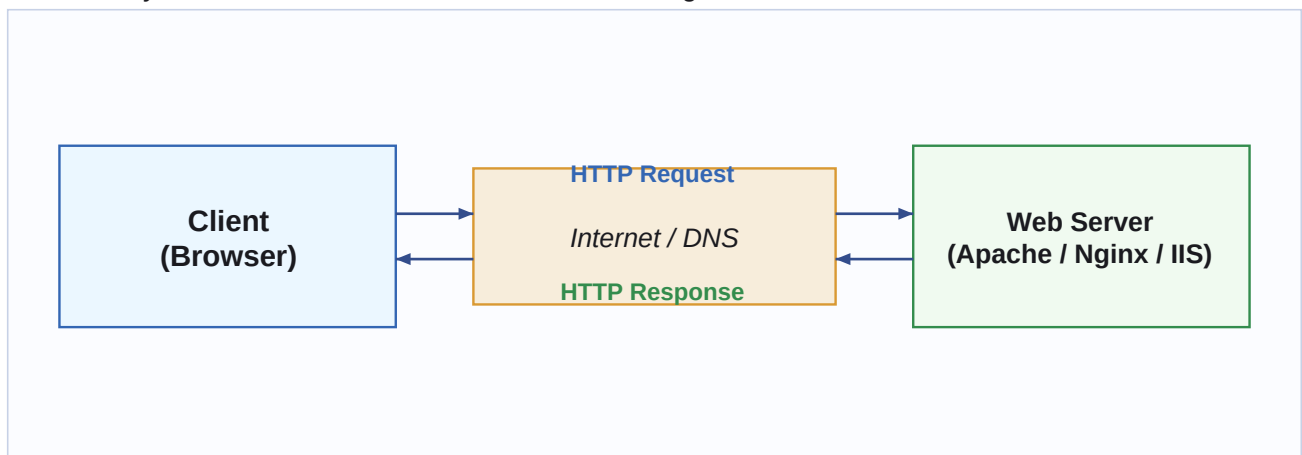


Figure 1.5 - Client / Server model of the Web

- **Protocols**- HTTP and HTTPS for web pages, FTP for file transfer, SMTP / IMAP / POP3 for mail, TCP / IP as the transport layer.
- **Web Browser** - software that fetches HTML over HTTP and renders it. Examples: Chrome, Firefox, Edge, Safari, Opera.
- **DNS (Domain Name System)** - the phone book of the Internet. It converts a human-readable name (www.tulas.edu.in) into an IP address (e.g. 142.250.183.4) that machines understand.
- **Web hosting provider**- a company that rents server space where your website's files are stored and served (GoDaddy, Hostinger, AWS, Bluehost, ...).

Q15. How do you set up a web server on Windows / Linux?

- **Windows - IIS** : Open *Control Panel - Programs - Turn Windows features on or off* and tick **Internet Information Services** Drop your HTML files into C:\inetpub\wwwroot and open http://localhost in a browser.
- **Windows - XAMPP** : Install XAMPP, start **Apache** from the control panel, place files in C:\xampp\htdocs and visit http://localhost.
- **Linux - Apache** : sudo apt install apache2, enable with sudo systemctl enable --now apache2, files go in /var/www/html.
- **Linux - Nginx** : sudo apt install nginx, sudo systemctl start nginx, files go in /var/www/html or /etc/nginx/sites-available/.

Q16. Explain types of web hosting (including cloud hosting).

- **Shared Hosting** - many websites share one server. Cheapest, but performance suffers if a neighbour gets busy.
- **VPS (Virtual Private Server)** - one physical server is split into several virtual ones; each user gets guaranteed RAM / CPU.
- **Dedicated Hosting** - the entire physical server is rented to one website. Most powerful, most expensive.
- **Cloud Hosting** - resources come from a pool of many servers (AWS, Azure, GCP). Pay-as-you-go and scales automatically.
- **Managed WordPress Hosting** - shared / cloud hosting tuned specially for WordPress with backups and security included.
- **Colocation** - you own the server hardware and rent only the rack space, power and bandwidth in a data centre.

Unit II - Questions and Answers

Q1. What is JavaScript? List its main features.

Answer

JavaScript (JS) is a high-level, interpreted, object-based scripting language designed to add **interactivity** to web pages. It runs inside the browser (client side) and, with Node.js, also on the server.

Features

- **Lightweight & interpreted**- no compilation step, the browser runs the code directly.
- **Dynamically typed**- variables don't have a fixed type (let x = 5; later x = "hi").
- **Event driven**- code reacts to user actions (click, key press, page load).
- **Object based**- everything is essentially an object; supports prototypes and ES6 classes.
- **Cross-platform**- runs on every browser and operating system.

Q2. Explain JavaScript functions and events with examples.

Functions

A **function** is a reusable block of code that performs a specific task. It is declared with the keyword **function** and can be called any number of times.

```
function add(a, b) {  
    return a + b;  
}  
alert(add(5, 3));    // 8  
  
// Arrow function (ES6)  
const square = n => n * n;  
console.log(square(7)); // 49
```

Events

An **event** is an action performed by the user or the browser - e.g. clicking a button, loading a page, pressing a key. JavaScript runs an **event handler** function when an event occurs.

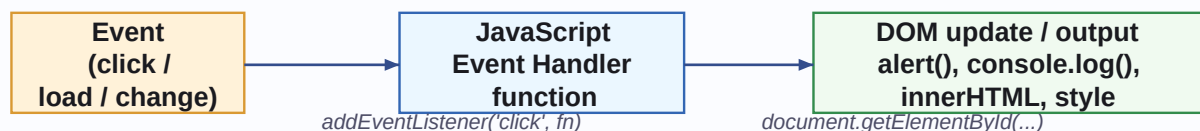


Figure 2.1 - JavaScript event handling pipeline

```
<button id="b1">Click me</button>

<script>
  document.getElementById("b1")
    .addEventListener("click", function () {
      alert("Button clicked!");
    });
</script>
```

Q3. What is the DOM? How do you traverse it using JavaScript?

Answer

The **Document Object Model (DOM)** is a tree-shaped, in-memory representation of an HTML document. Every tag becomes a **node**; JavaScript can read, change, add or remove these nodes to update the page on the fly.

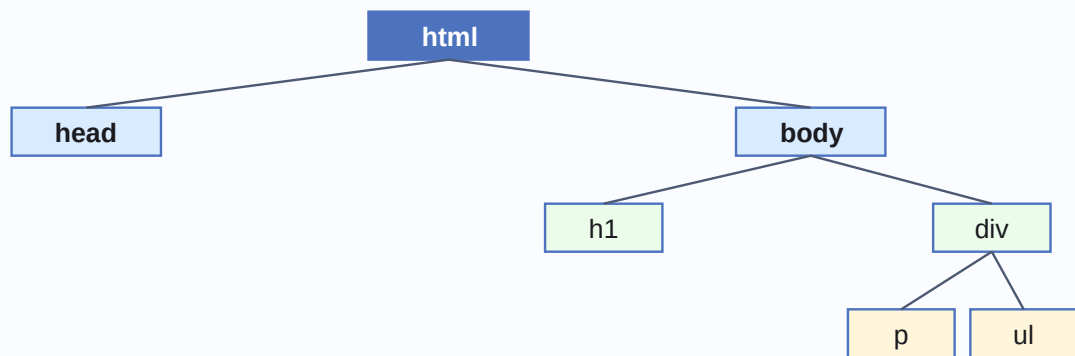


Figure 2.2 - A simple DOM tree

Common traversal / selection methods

- `document.getElementById("id")` - one element by id.
- `document.getElementsByTagName("c")` - HTMLCollection.
- `document.querySelector("css")` - first matching element.
- `document.querySelectorAll("css")` - NodeList of all matches.
- `element.parentNode`, `element.children`, `element.firstChild`, `element.nextElementSibling` - move around the tree.

```
// change the text of <h1 id="title">
let h = document.getElementById("title");
h.textContent = "Welcome, Aizaz!";
h.style.color = "crimson";

// loop over all paragraphs in <div id="box">
let box = document.querySelector("#box");
for (let p of box.children) {
  console.log(p.textContent);
}
```

Q4. Explain the output system in JavaScript: alert, prompt, console.log, document

- `alert(msg)` - shows a pop-up dialog with msg and an OK button. Used for quick debugging only.

- **prompt(msg, default)**- pops up an input box; returns the string typed (or null if Cancel).
- **confirm(msg)**- shows OK / Cancel; returns true / false.
- **console.log(...)** - prints to the **browser developer console** Most professional way to inspect values.
- **document.write(html)**- writes HTML directly into the document. Avoid in production - it overwrites the page after load.

```
let name = prompt("What is your name?", "Aizaz");
if (confirm("Are you a BCA student?")) {
    alert("Welcome " + name + " to BCA!");
    console.log("User:", name);
} else {
    document.write("<h2>Sorry, this page is for BCA.</h2>");
}
```

Q5. Explain variables and arrays in JavaScript.

Variables

- **var** - old way, function-scoped, can be re-declared.
- **let** - modern, block-scoped, cannot be re-declared in the same block.
- **const** - block-scoped constant. The reference cannot be reassigned.

```
var   name   = "Aizaz";
let   age    = 21;
const PI     = 3.14159;

age = 22;           // OK
// PI = 3.14;       // TypeError
```

Arrays

An **array** is an ordered, zero-indexed list that can hold values of any type. It is itself an object with many useful methods.

```
let subjects = ["WT", "DBMS", "OS", "DSA"];
console.log(subjects.length); // 4
subjects.push("AI");          // add to end
subjects.pop();               // remove last
subjects.forEach(s => console.log(s));

let mixed = [1, "two", true, [3, 4], { k: "v" }];
```

Q6. Explain Date and String handling in JavaScript.

Date object

```
let now = new Date();
console.log(now.toString()); // Mon May 18 2026
console.log(now.getFullYear()); // 2026
console.log(now.getMonth() + 1); // 5 (0-based!)
console.log(now.getHours() + ":" + now.getMinutes());

let dob = new Date("2003-11-12");
let age = (now - dob) / (1000*60*60*24*365.25);
console.log("Age:", Math.floor(age));
```

String methods

- **length** - number of characters.
- **toUpperCase()** / **toLowerCase()** - change case.
- **indexOf(s)** / **includes(s)** - search.
- **slice(a, b)** / **substring(a, b)** - extract a portion.
- **replace(a, b)** - replace first match.
- **split(sep)** / **Array.join(sep)** - convert between strings and arrays.
- **trim()** - remove leading / trailing whitespace.

```
let s = " Web Technology ";
console.log(s.trim().toUpperCase()); // "WEB TECHNOLOGY"
console.log(s.includes("Tech")); // true
console.log("Aizaz Ahmad".split(" ")); // ["Aizaz", "Ahmad"]
```

Q7. How can JavaScript manipulate CSS at runtime?

JavaScript can change any CSS property of an element through its **style** object or by adding / removing classes via **classList**

```
let btn = document.getElementById("go");

// 1) inline style
btn.style.backgroundColor = "#0d6efd";
btn.style.color          = "white";
btn.style.padding        = "10px 20px";

// 2) toggle a class
btn.classList.add("active");
btn.classList.toggle("hidden");
```

Q8. Write a JavaScript form-validation example covering required, length and pattern

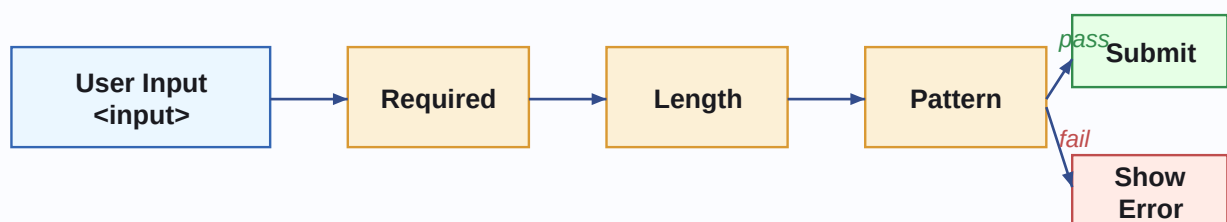


Figure 2.3 - Validation pipeline (required / length / pattern)

```

<form id="reg" onsubmit="return validate()">
  Name : <input id="name"> <br>
  Pwd : <input id="pwd" type="password"> <br>
  Email: <input id="email"> <br>
  <input type="submit" value="Register">
</form>

<script>
function validate() {
  let name = document.getElementById("name").value.trim();
  let pwd = document.getElementById("pwd").value;
  let email = document.getElementById("email").value;

  // 1. required
  if (name === "") { alert("Name is required"); return false; }

  // 2. length
  if (pwd.length < 6) { alert("Password >= 6 chars"); return false; }

  // 3. pattern (regex)
  let re = /^[A-Za-z0-9._]+@[A-Za-z]+\.[A-Za-z]{2,}$/;
  if (!re.test(email)) { alert("Invalid email"); return false; }

  return true; // all checks passed -> form is submitted
}
</script>

```

Q9. How does JavaScript control the browser? Give 5 useful objects / methods.

- **window.open(url) / window.close()** - open or close a browser window or tab.
- **window.location.href = "..."** - navigate to another page; **location.reload()** refreshes the current page.
- **window.history.back() / forward()** - move through the browser history.
- **navigator.userAgent, navigator.language, navigator.onLine** - information about the browser and connection.
- **screen.width / screen.height** - size of the user's display.
- **setTimeout(fn, ms) / setInterval(fn, ms)** - schedule a function once or repeatedly.

```

console.log(navigator.userAgent);
console.log("screen:", screen.width + "x" + screen.height);

setTimeout(() => alert("Hello after 2 sec"), 2000);

document.getElementById("go").onclick = () => {
  window.location.href = "https://www.tulas.edu.in";
};

```

Q10. What is AJAX? Explain its working with a diagram.

Answer

AJAX (Asynchronous JavaScript And XML) is a technique that lets a web page exchange small amounts of data with a server **in the background** so only a part of the page is updated - the user does not see a full reload. Although the name says XML, today most AJAX requests carry **JSON**.

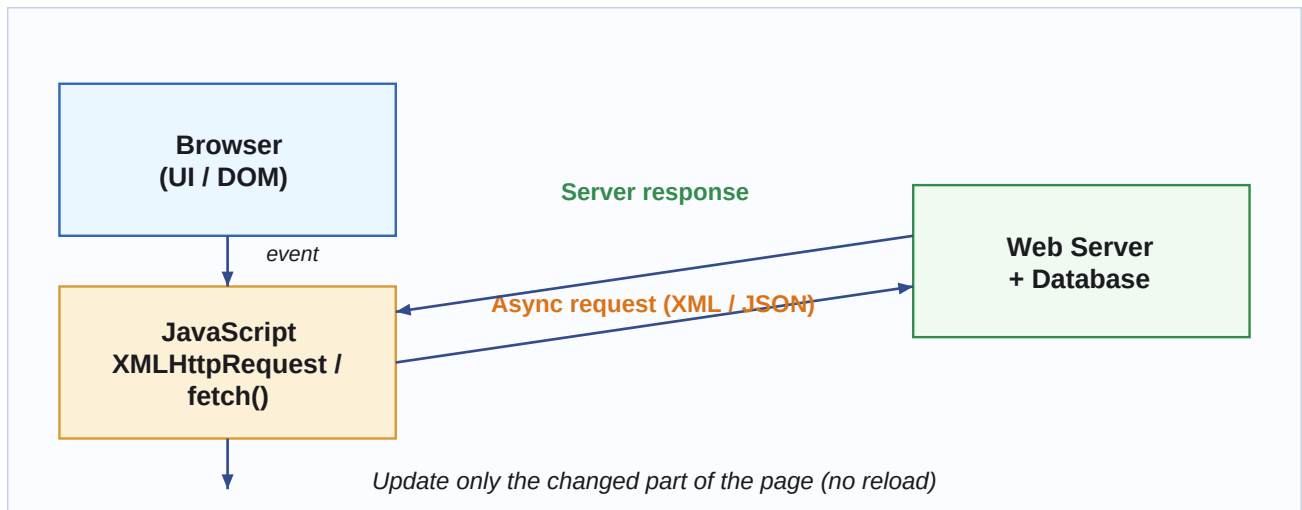


Figure 2.4 - How an AJAX request flows

Advantages

- **Faster, smoother UX** - no full-page refresh.
- **Less bandwidth** - only the changed data is sent.
- **Better separation** of UI and data.
- **Asynchronous** - the page remains usable while waiting.

Disadvantages

- **Won't work if JavaScript is disabled.**
- **Browser back button** may not work as users expect.
- **Same-Origin Policy** - cross-domain requests need CORS or JSONP.
- **Security** - all client code is visible; data must still be validated on the server.

Alternatives to AJAX

- **Fetch API** - modern, promise-based replacement for XMLHttpRequest.
- **WebSockets** - full-duplex, real-time channel (chat, live updates).
- **Server-Sent Events (SSE)** - one-way push from server to browser.
- **GraphQL / gRPC-Web** - newer query / RPC styles over HTTP.

```

// Classic XHR
let xhr = new XMLHttpRequest();
xhr.open("GET", "/api/students");
xhr.onload = () => {
  if (xhr.status === 200)
    document.getElementById("out").innerHTML = xhr.responseText;
};
xhr.send();

// Modern fetch
fetch("/api/students")
  .then(r => r.json())
  .then(d => console.log(d))
  .catch(e => console.error(e));
  
```

Q11. What is XML? Differentiate XML and HTML.

Answer

XML (eXtensible Markup Language) is a markup language designed to **store and transport data** in a

self-describing, platform-independent way. Unlike HTML, the tags in XML are **user-defined** and carry only data, not presentation.

Key concepts

- Every XML file has exactly one **root element**
- Tags are **case sensitive** and must be **properly nested** and **closed**.
- Attributes** are written in quotes: <book id="1">.
- XML is well-formed** if it follows these syntax rules; it is **valid** if it also matches a DTD or XML Schema.

HTML vs XML (table)

Feature	HTML	XML
Purpose	Display data	Store / transport
Tags	Predefined	User-defined
Case sensitive	No	Yes
Closing tag	Optional sometimes	Mandatory
Whitespace	Ignored	Preserved

```
<?xml version="1.0" encoding="UTF-8"?>
<students>
  <student id="101">
    <name>Aizaz Ahmad</name>
    <course>BCA</course>
    <marks>92</marks>
  </student>
  <student id="102">
    <name>Ravi Kumar</name>
    <course>BCA</course>
    <marks>88</marks>
  </student>
</students>
```

Q12. What are DTD and XML Schema (XSD)?

- DTD (Document Type Definition)** - the older way to describe the structure (allowed elements, order, attributes) of an XML document.
- XSD (XML Schema Definition)** - the newer, more powerful alternative. It is itself written in XML and supports data types (string, int, date) and namespaces.

```
<!-- student.dtd -->
<!ELEMENT students (student+)>
<!ELEMENT student (name, course, marks)>
<!ATTLIST student id CDATA #REQUIRED>
<!ELEMENT name (#PCDATA)>
<!ELEMENT course (#PCDATA)>
<!ELEMENT marks (#PCDATA)>
```



```

<!-- student.xsd (XML Schema) -->
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="student">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="name" type="xs:string"/>
        <xs:element name="course" type="xs:string"/>
        <xs:element name="marks" type="xs:int"/>
      </xs:sequence>
      <xs:attribute name="id" type="xs:int" use="required"/>
    </xs:complexType>
  </xs:element>
</xs:schema>

```

Q13. Explain XSL, XSLT and the transformation process.

Answer

XSL (eXtensible Stylesheet Language) is a family of languages for working with XML. **XSLT (XSL Transformations)** is the part used to **transform an XML document into another format** usually HTML, plain text, PDF or even another XML.

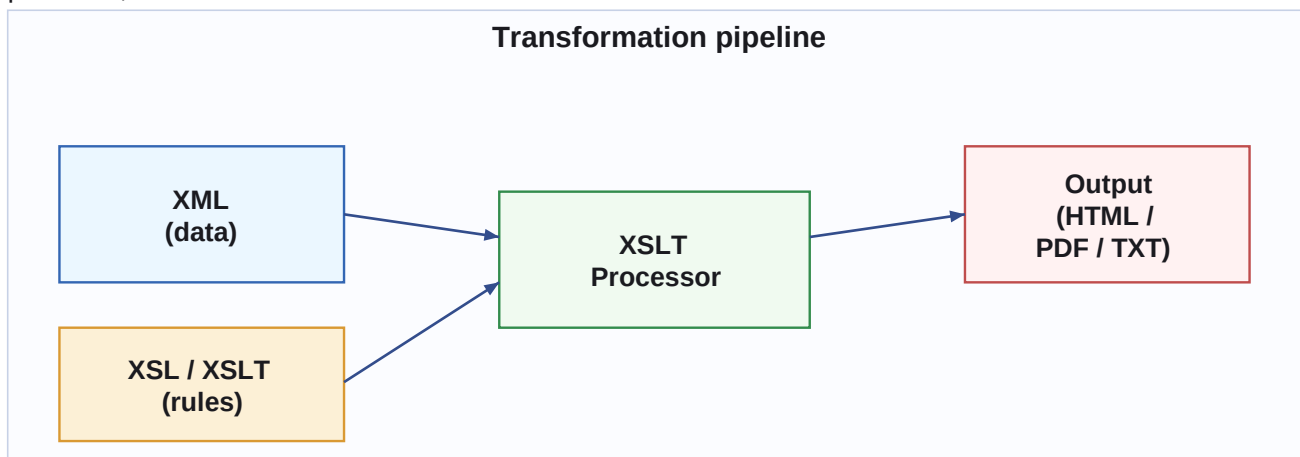


Figure 2.5 - XML + XSLT transformation pipeline

Important XSL elements

- **<xsl:stylesheet>** - the root of every XSLT document.
- **<xsl:template match="...">** - rule that fires when an XPath pattern matches.
- **<xsl:value-of select="...">** - outputs the value of a node.
- **<xsl:for-each select="...">** - loops over a node set.
- **<xsl:if>, <xsl:choose> / <xsl:when>** - conditional logic.

```
<?xml version="1.0"?>
<xsl:stylesheet version="1.0"
  xmlns:xsl="http://www.w3.org/1999/XSL/Transform">

<xsl:template match="/students">
  <html><body>
    <h2>Student List</h2>
    <table border="1">
      <tr><th>Name</th><th>Course</th><th>Marks</th></tr>
      <xsl:for-each select="student">
        <tr>
          <td><xsl:value-of select="name"/></td>
          <td><xsl:value-of select="course"/></td>
          <td><xsl:value-of select="marks"/></td>
        </tr>
      </xsl:for-each>
    </table>
  </body></html>
</xsl:template>
</xsl:stylesheet>
```

Q14. What is XHTML? How does it differ from HTML?

XHTML (eXtensible HyperText Markup Language) is HTML rewritten as a **well-formed XML application**. It looks almost identical to HTML but follows the strict syntax rules of XML.

- All tags must be **lowercase** (<p>, not <P>).
- Every tag must be **closed**, including empty tags:
, .
- Tags must be **properly nested** <i>hi</i>, not <i>hi</i>.
- All attribute values must be **quoted** ().
- The document must start with an **XHTML DOCTYPE** and an **xmlns** attribute on <html>.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE html PUBLIC
  "-//W3C//DTD XHTML 1.0 Strict//EN"
  "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
  <head><title>Demo</title></head>
  <body><p>Hello, XHTML.</p></body>
</html>
```

Q15. What is JSON? Explain its keys, values and types with examples.

Answer

JSON (JavaScript Object Notation) is a lightweight, text-based data-interchange format. It is **language independent**, easy for humans to read and write and easy for machines to parse and generate. Today JSON has almost completely replaced XML in REST APIs and AJAX.

Structure

- **Object** - a collection of unordered key:value pairs wrapped in curly braces { }.
- **Array** - an ordered list wrapped in square brackets [].
- **Key** - always a **string in double quotes**
- **Value** - one of: string, number, boolean, null, array or another object.

key	value	type
name	"Aizaz"	string
age	21	number
isStudent	true	boolean
subjects	["WT", "DBMS"]	array
addr	{ "city": "Dehradun" }	object
score	null	null

Figure 2.6 - JSON object: keys, values and their data types

```
{
  "name":      "Aizaz Ahmad",
  "age":       21,
  "isStudent": true,
  "score":     null,
  "subjects":  ["WT", "DBMS", "OS"],
  "address": {
    "city":    "Dehradun",
    "state":   "Uttarakhand"
  }
}
```

Working with JSON in JavaScript

```
let jsonStr = '{"name":"Aizaz","age":21}';
let obj     = JSON.parse(jsonStr);    // string -> object
console.log(obj.name);                // "Aizaz"

let back    = JSON.stringify(obj);    // object -> string
console.log(back);
```

Q16. Differentiate JSON and XML in tabular form.

Feature	JSON	XML
Syntax	Lightweight, braces	Verbose, tags
Data types	String, Number, Boolean, null, Object, Array	Everything is text (typing via XSD)
Parsing in JS	JSON.parse() - native	Need DOMParser
File size	Smaller	Larger
Comments	Not allowed	Allowed
Best for	REST APIs, configs	Document storage, config with schema

End of Question Bank

These notes were prepared by **Aizaz Ahmad** of **Tulas Institute, Department of Computer Application** to help students of BCA / MCA quickly revise the **Web Technology** syllabus (Units I and II).

Quick exam tips

1) Always start a question with a one-line definition. 2) Support every theory answer with a small code example. 3) For 5 / 10-mark questions add a labelled diagram. 4) For comparison questions, use a 2-column table. 5) Memorise the syntax of one tag / one selector / one JS object per topic - examiners love a clean example.

All the best for the examination!